

Artificial Intelligence

Fiona Yan Liu

Department of Computing

The Hong Kong Polytechnic University

Learning Outcomes of Lecture 02

- Understand important concepts of knowledge based agent
- Master how to query using forward chaining and backward chaining
- Understand the definition of entropy and information gain
- Master how to build a decision tree based on the given data

Agenda

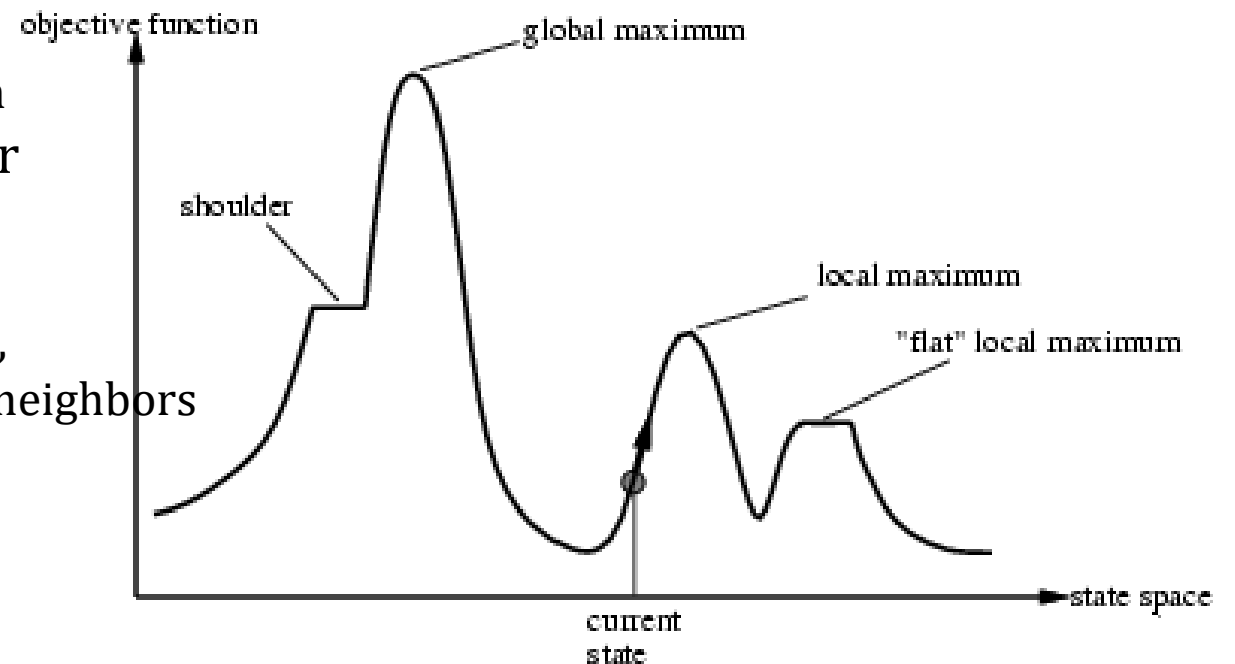
- Landscape of search
- Hill-climbing search
- Simulated annealing search
- Genetic algorithm

Agenda

- Landscape of search
- Hill-climbing search
- Simulated annealing search
- Genetic algorithm

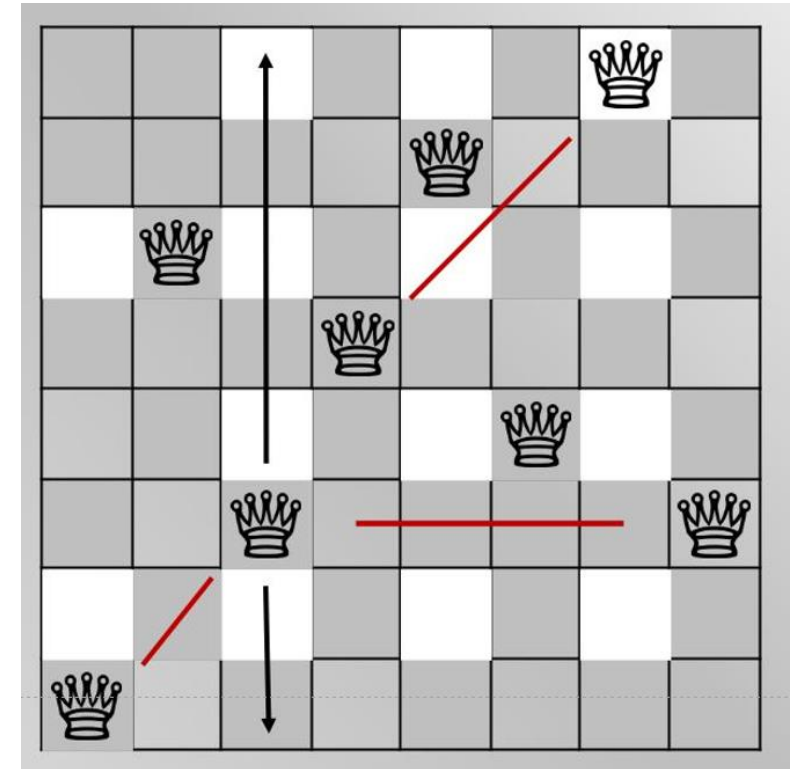
“Landscape” of Search

- Horizontal axis (state space)
 - Represents all possible solutions or states.
- Vertical axis (objective function)
 - Represents the value of the objective function, which we want to maximize or minimize.
- The algorithms discussed in heuristics search
 - To find a goal state from a start state
 - the path to the goal is a solution to the search
- In many problems the path doesn't matter
 - the goal state itself is the solution
- Local search algorithms
 - at each step consider a single “current ” state,
 - and try to improve it by moving to one of its neighbors
- → Iterative improvement algorithms



Example: n-queens

- Put n queens on an $n \times n$
- no two queens on the same row, column, diagonal, or anti-diagonal
- Example of a heuristic function $h(n)$
 - the number of pairs of queens that are attacking each other
- Minimize the number of pairs of queens



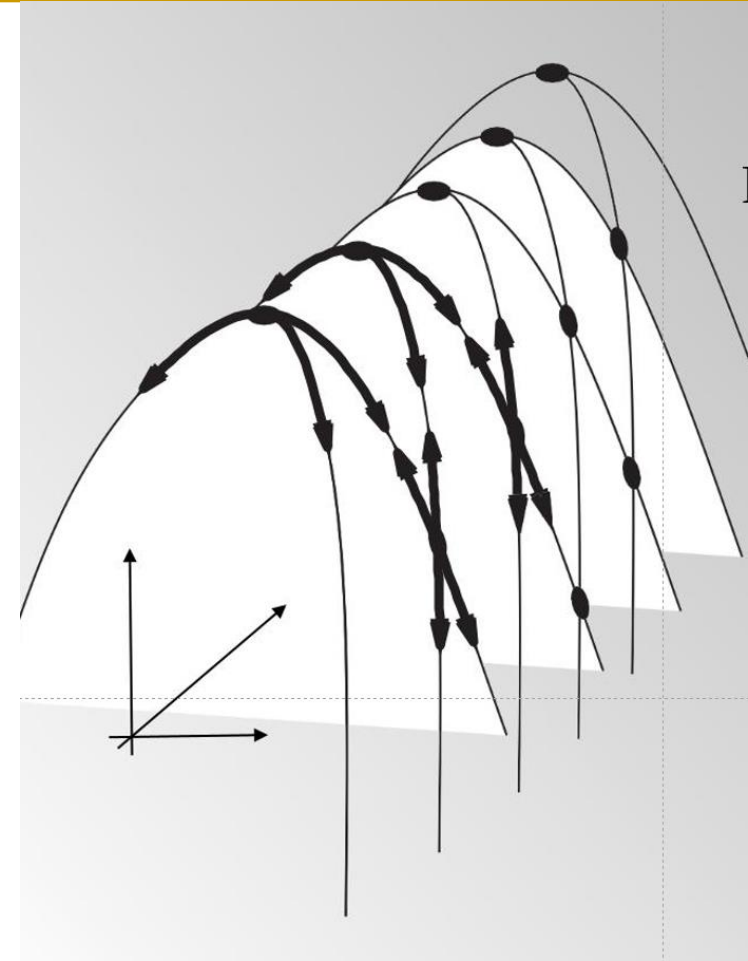
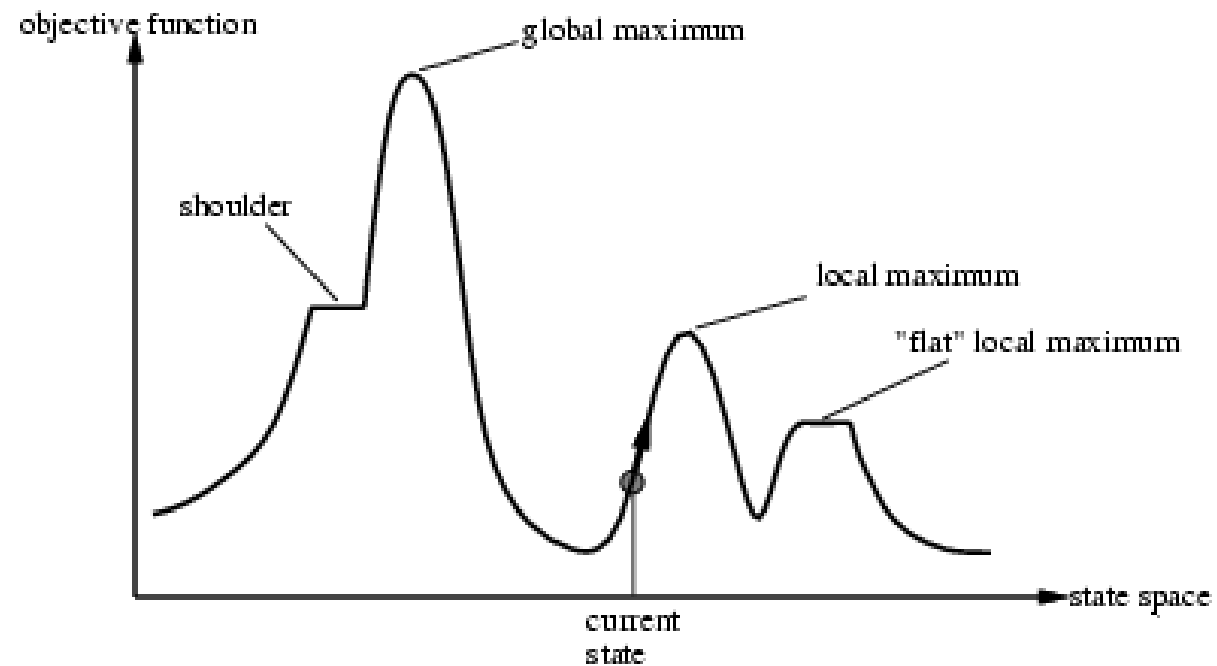
Agenda

- Landscape of search
- Hill-climbing search
- Simulated annealing search
- Genetic algorithm

Hill-climbing search: 8-queens problem

- Convert to an optimization problem
 - h = number of pairs of queens that are attacking each other
 - $h = 17$ for the below state
- Randomly generated 8-queens starting states
 - 14% chance: it solves the problem
 - 86% chance: it get stuck at a local minimum

HILL-CLIMBING CAN GET STUCK



Agenda

- Landscape of search
- Hill-climbing search
- Simulated annealing search
- Genetic algorithm

Simulated Annealing Search

- Annealing
 - physical process of cooling a liquid or metal
 - until particles achieve a certain frozen crystal state
- Simulated Annealing
 - physics inspired twist on random walk
- Escape from local optima
 - by accepting, with a probability that decreases during the search
 - also moves that are worse than the current solution
- Compared with hill climbing search
 - going downhill



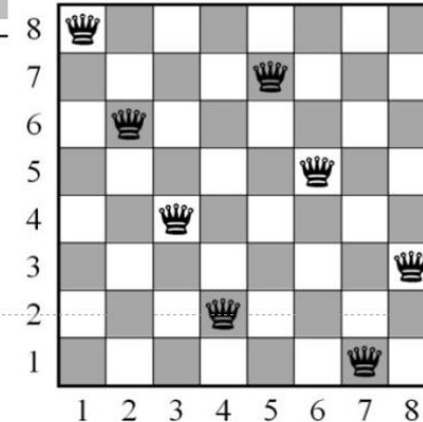
Rationale of Simulated Annealing Search

- Identify the quality of the local improvements
 - instead of picking the best move, pick one randomly
 - say the change in objective function is d
 - if d is positive
 - then move to that state
 - otherwise:
 - move to this state with probability proportional to d
 - thus: worse moves (very large negative d) are executed less often
- There is always a chance of escaping from local maxima
 - over time, make it less likely to accept locally bad moves

Solving 8-queen Problem using SA

- Each state is defined with an objective function f
- Randomly check which move can have better f
- Accept the move with a probability

```
 $T = T_{max};$  /* Initializing the temperature */  
 $s = s_0;$  /* initial solution */  
Best solution =  $s$   
repeat  
  Generation of a random neighbor:  $s'$   
  If  $\Delta E = f(s') - f(s) \leq 0$   
    Then  $s = s';$  /* Accept the neighbor */  
    Else Accept  $s'$  with a probability of  $e^{-(\Delta E/T)}$  and set  $s = s'$   
  Best solution =  $s$   
   $T = g(T);$  /* Temperature update according to the cooling schedule */  
Until Stopping criteria satisfied  
Return Best solution
```



Note: this algorithm is **minimizing** the objective function

Agenda

- Landscape of search
- Hill-climbing search
- Simulated annealing search
- Genetic algorithm

Rationale of Genetic Algorithm

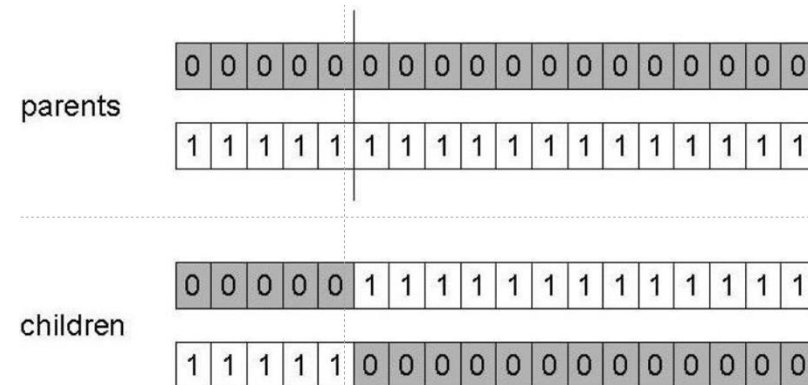
- A particular class of evolutionary algorithms
 - that use techniques inspired by evolutionary biology such as inheritance, mutation, selection, and crossover
- The evolution usually starts from
 - a population of randomly generated individuals and happens in generations.
- In each generation
 - The fitness of every individual in the population is evaluated
- Multiple individuals are selected from the current population
 - based on their fitness
 - and modified (recombined and possibly mutated) to form a new population

Genetic Algorithm

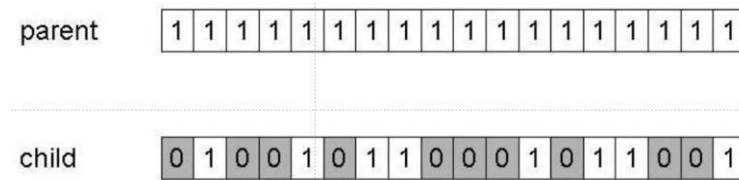
- Twist on Local Search
 - successor is generated by combining two parent states
 - A state is represented as a string over a finite alphabet
 - 8-queens
 - State = position of 8 queens each in a column
- Start with k randomly generated states (population)
- Evaluation function (objective/fitness function)
 - Higher values for better states
 - Opposite to heuristic function, e.g., # non-attacking pairs in 8-queens
- Produce the next generation of states by “simulated evolution”
 - Random selection
 - Crossover
 - Random mutation

Crossover and Mutation

- Probability of crossover P_c
 - Split parents at this crossover point



- The mutation rate P_m
 - Alter each gene independently with a probability



Apply GA to 8-queens

- String representation
- 16257483

